

Confirmation-Gated HIL Fault-Injection Trials for Auditable Payload-Link Supervision

Abstract—An injection request does not show that the injected endpoint entered the configured fault mode. This distinction matters when controller detection and recovery are scored from separate observations. We present a confirmation-gated hardware-in-the-loop (HIL) trial protocol for a supervised payload link. Payload-side confirmation of configured fault-mode entry gates detection scoring, and payload-side confirmation of NORMAL restoration gates recovery scoring. These two gates are operationalized as distinct requested, activated, detected, restored, and recovered events. Byte-preserving controller and payload traces and hash-verified summaries support auditability. In a setup-bounded demonstration, one G431/G474 pair completed 90 sequential trials, 30 each for silence, bad CRC, and delay. All 90 trials had confirmed activation and restoration; the predefined detector and recovery markers appeared in 30/30 valid trials per mode. Host-observed median command-to-marker intervals ranged from 359.5 to 406.0 ms for detection and from 296.5 to 359.0 ms for recovery. Separate evidence comprises a 605-s nominal observation, two BAD_CRC mechanism observations, and two 12-row F411 pair campaigns reported with separate denominators. The results demonstrate the protocol on the evaluated setups. They are not reliability, device-population, MCU-equivalence, or controller-internal timing estimates.

Index Terms—hardware-in-the-loop, fault injection, embedded systems, payload link, reproducibility

I. INTRODUCTION

Embedded controllers often supervise whether a peripheral responds, whether a received frame is valid, and whether an unavailable peer returns to service. HIL fault injection can exercise these paths, but its evidence is ambiguous if a sent injector command is treated as proof that the endpoint entered the configured fault mode. The same problem recurs at recovery: requesting restoration does not show that the endpoint returned to NORMAL mode. Controller markers should be scored only after those endpoint mode transitions are separately confirmed.

Fault-injection methodology distinguishes faults, functional activation, readouts, and measures [1]. Martínez *et al.* confirm effective pin-level injection and filter invalid event sequences [2]. Automotive HIL and nanosatellite studies provide programmatic faults, communication-bus interception, multi-point observation, logging, and test traceability [3], [4], [5], [6]. The gap addressed here is therefore not activation verification itself. Rather, in the closest studies considered here, the cited methods do not specify a stage-specific scoring rule that conditions detection on separate confirmation that the endpoint entered the configured fault mode and recovery on separate confirmation that the endpoint returned to NORMAL mode.

We therefore propose a two-stage confirmation-gated experimental protocol. A command is a request, not confirma-

tion of an endpoint mode transition. Separate payload-side mode confirmation establishes configured fault-mode entry or restoration to NORMAL; only then can a controller-side detector or recovery marker be scored. The five event classes operationalize these gates. Host timestamps, exact received bytes, readable renderings, and hash-verified frozen summaries support auditability.

The paper's one central methodological contribution is the two-stage confirmation-gated scoring rule: payload-side confirmation of configured fault-mode entry makes detection eligible, and payload-side NORMAL confirmation makes recovery eligible. The distinct requested, activated, detected, restored, and recovered events operationalize this rule. The byte-preserving, hash-verified evidence workflow implements its audit support, and the setup-bounded campaigns evaluate its operation; neither is claimed as a separate research contribution.

The evaluated firmware and boards are context, not an architecture contribution or a new HIL framework, injector, simulator, or fault model. The study does not claim comparative advantage, reliability, population generality, flight qualification, mission assurance, MCU-family equivalence, or complete-system validation.

II. RELATED WORK

Foundational fault-injection methodology distinguishes simulation- and prototype-based evaluation, including hardware- and software-implemented techniques, and couples controlled injection with monitoring of system effects [7]. Work on fault-injection validity further separates requested faults from realized effects. Arlat *et al.* define fault, functional-activation, readout, and measure sets (FARM) [1]. Their activation set functionally exercises the system; separate endpoint confirmation that a requested condition became active is not reported in the cited publication. Martínez *et al.* use AFIT for high-speed pin-level injection and distinguish Activated Injection from Effective Error; an injector circuit asserts the latter only when the selected FTS pin is forced [2]. They discard non-effective injections, glitches, and invalid event sequences before analysis and measure detection latency from Effective Error. A separately observed restoration-to-normal gate before recovery scoring is not reported in the cited paper. Van der Kouwe *et al.* distinguish inserted faults as an input fault load from the subset activated during execution as the output fault load when analyzing software-injection distortion [8]. Payload-endpoint confirmation and restoration gates are not applicable to that study design.

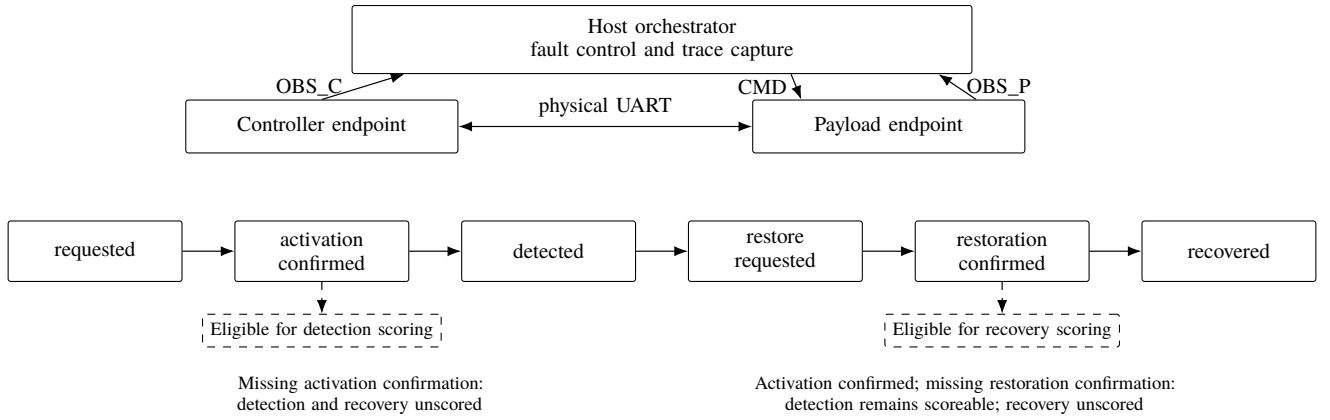


Fig. 1. Confirmation-gated topology and event sequence. Payload-side confirmation determines scoring eligibility; missing restoration confirmation leaves an already eligible detection scoreable and recovery unscored. OBS_C and OBS_P are captured by the host orchestrator. Timeline arrows denote scoring stages, not controller states or strict receipt order.

Automated HIL frameworks address configurable fault campaigns and system-level observation. Short *et al.* combine statistical fault-injection testing with online model-based performance monitoring in an automotive HIL methodology and compare eight embedded-control designs in a case study [9]. Abboush *et al.* configure fault type, location, time, and duration in real-time automotive HIL experiments and record multiple system signals for evaluation [3]. Their later framework automates campaigns across virtual driving scenarios, including multiple fault locations, and reports fault deactivation followed by return toward a safe state [4]. Separate injected-endpoint confirmation before detection scoring and separate restoration confirmation before recovery scoring are not reported in the cited Abboush publications.

Communication-level and nanosatellite work addresses physical injection, bus interception, and multi-point observation. Blanc *et al.* apply pin-level physical injection to TTP/C communication controllers, including faults induced directly on communication lines and data errors at the controller-host interface [10]. Batista *et al.* present FEM as a scripted interceptor for MIL/HIL nanosatellite robustness testing [5]. FEM observes both low interfaces, returns exchanged-message data to its controller, and reports transmitted and received values and delays in MIL test tables. Only a small HIL set was attempted before testing was suspended after interface mismatches were found. Interface observation is reported, but per-trial confirmation gates for detector or recovery score eligibility are not reported in the cited paper. Conceição and Mattiello-Francisco organize interoperability testing into six MIL, SIL, and HIL modules, reuse model-derived tests, and use FEM for bus-fault tests [6].

Together, these studies establish activation or effective-injection confirmation, invalid-sequence filtering, communication-bus injection, multi-point observation, and test traceability as prior art. The central distinction here is only the combined two-stage scoring rule: separate payload-side confirmation of configured fault-mode entry gates

detection, and separate payload-side NORMAL confirmation gates recovery. The five retained events operationalize those gates, while exact-byte traces and hashes support auditability. The protocol complements rather than replaces AFIT-style filtering, general HIL architectures, FEM, or SATS. “Not reported in the cited paper” is limited to that publication and does not imply that a system lacks the feature.

Table I uses three statuses. *Reported* identifies an element stated in the cited paper. *Not reported in the cited paper* records only that the publication does not describe the element. *Not applicable* marks a comparison that presupposes an endpoint or scoring phase outside the study design; it is not an absence claim. Only the detection-eligibility and recovery-eligibility columns establish the claimed literature gap. The other columns provide comparison context and describe operationalization or audit support; they do not establish additional novelty claims.

III. SYSTEM CONTEXT AND PROTOCOL

A. Evaluated Link

The primary setup connects a NUCLEO-G431RB controller to a NUCLEO-G474RE payload simulator through a 115200-baud, 8N1 UART with common ground. The controller polls the payload and exposes timeout, rejection, OFFLINE, and recovery markers. The commandable payload returns NORMAL responses, withholds responses in SILENT mode, corrupts response CRCs in BAD_CRC mode, and delays responses by 250 ms in DELAYED mode. The inspected controller deadline is 100 ms.

As Fig. 1 shows, the host sends fault-control commands and separately captures controller (OBS_C) and payload (OBS_P) serial streams. These are separate observation channels collected on the same host; no stronger host or statistical independence is implied. The topology is the minimum context needed to define the protocol.

TABLE I
COMPARISON WITH SELECTED CLOSEST WORK; ONLY THE TWO ELIGIBILITY-GATE COLUMNS ESTABLISH THE CLAIMED GAP.

Study	FI setting	Input/request vs. activation	Detection eligibility gate	Recovery eligibility gate	Five-event operationalization	Audit-supporting evidence
Arlat/FARM [1]	Reported: physical FI method	Reported: F , functional-exercise A , R , and M	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper
Martínez/AFIT [2]	Reported: pin-level physical FI	Reported: Activated Injection vs. Effective Error	Reported: non-effective trials removed before detection measures	Not reported in the cited paper: no observed normal-restoration gate	Reported: activation/effective-error/detection/recovery; request/restored: not reported in the cited paper	Reported: analyzer and injector logs; exact-byte/hash: not reported in the cited paper
Van der Kouwe [8]	Reported: software FI	Reported: input and activated output fault loads	Not applicable: no payload endpoint	Not applicable: no restoration phase	Not applicable: no endpoint event chain	Not applicable: no endpoint-byte evidence
Abboush 2022 [3]	Reported: automotive HIL FI	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper
Abboush 2024 [4]	Reported: automotive HIL FI	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper
Batista/FEM [5]	Reported: bus MIL/HIL FI	Reported: script vs. monitored Tx/Rx outcomes	Not reported in the cited paper: no detector eligibility gate	Not reported in the cited paper	Not reported in the cited paper	Reported: serial and Tx/Rx tables; exact-byte/hash: not reported in the cited paper
Conceição/SATS [6]	Reported: bus MIL/SIL/HIL FI	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper	Not reported in the cited paper
This work	Reported: UART HIL FI	Reported: request and confirmed fault-mode entry	Reported: stage-1 payload gate	Reported: stage-2 NORMAL gate	Reported: explicit five-event operationalization	Reported: exact bytes and SHA-256 support auditability

B. Validity and Scoring

For trial i , let $A_i, S_i, E_i \in \{0, 1\}$ indicate, respectively, payload confirmation that the endpoint entered the configured fault mode, payload confirmation that it returned to NORMAL, and the presence of a harness invalid reason. Define the complete-trial validity indicator as

$$V_i = A_i \wedge S_i \wedge \neg E_i. \quad (1)$$

Detection eligibility remains stage-specific: without A_i , the trial enters neither the detection nor recovery denominator. If $A_i = 1$ but $S_i = 0$, detection remains scoreable while recovery is unscored. This stage-specific rule does not change the primary results because all 90 primary trials contained both confirmations.

The payload firmware assigns the selected mode and configured delay, when applicable, before emitting the matching mode-confirmation line. That confirmation therefore establishes the scorer’s causal eligibility boundary but is not an independent physical measurement of UART behavior. Because OBS_C and OBS_P are separate host-captured serial channels without a verified total order, a controller marker recorded before matching activation confirmation makes the cross-channel detection ordering temporally ambiguous. Detection is then left unscored rather than classified as DETECTED or NOT_DETECTED; this ordering ambiguity does not by itself invalidate the trial or alter independently eligible recovery scoring.

To exercise the stage-specific branches without changing the HIL records, we derived five explicitly synthetic event-sequence mutations from valid BAD_CRC trial R005_P02 and replayed them through a separate scorer. Missing or

wrong-mode activation confirmation left detection and recovery unscored; missing restoration confirmation after valid activation left detection scored and recovery unscored. A controller marker assigned a host timestamp before matching payload confirmation produced temporally ambiguous cross-channel ordering and left detection unscored, while recovery remained independently scoreable; a duplicate matching activation confirmation was flagged and counted once. The mutation manifest, expected and actual outputs, assertions, and source and derived SHA-256 records are retained separately, and all checked source hashes remained unchanged. These cases check scorer control flow only; they are not HIL trials and are excluded from the 90-trial and all other empirical denominators.

The executed sequence comprised establishing and confirming a NORMAL precondition, waiting a seeded offset, requesting a fault mode, requiring the matching payload confirmation, observing the controller for 4 s, requesting NORMAL, requiring payload confirmation of NORMAL, and observing recovery for 3 s. Seed 20260830 fixed the order and offsets of 90 sequential primary trials, 30 per mode.

Mode-specific detector predicates were fixed before analysis. SILENT and DELAYED use the first post-request timeout or OFFLINE marker. BAD_CRC uses the first explicit CRC-rejection marker; OFFLINE is secondary. Recovery requires a recovery marker after the NORMAL request and confirmed restoration. A restart field records only whether a post-injection link-start marker appears; absence of that marker is not proof that no reset occurred.

At the evaluated controller revision, a CRC-invalid response increments the CRC-rejection counter and returns without

TABLE II
VALIDATED NOMINAL OBSERVATIONS.

Phase	s	Status	ok first-last	TO	CRC	SEQ	REC
Pre	65	13	1835–1955	0	0	0	0
Post	65	13	3049–3169	0	0	0	0
Separate	605	121	1760–2960	0	0	0	0

clearing the outstanding request. The 100-ms deadline can then expire and increment a consecutive-timeout count; the third consecutive timeout sets OFFLINE. A later accepted response clears the request and timeout streak, sets ONLINE, and records recovery if the prior state was OFFLINE. These source-level statements explain marker order but do not measure MCU execution time.

C. Observation and Audit Model

Each raw serial record contains host UTC time, exact received bytes, and an escaped readable rendering. Controller and payload files remain separate. During acquisition, the harness computes detection latency from host-monotonic times for the fault-command send and the first predefined controller marker. Recovery latency is computed analogously from the NORMAL-command send and the recovery marker and is populated only with restoration confirmation. The result rows retain these computed intervals, while raw logs retain UTC and exact bytes. Both intervals include UART, USB, driver, scheduling, and host timestamp effects; host and MCU clocks were not synchronized.

Machine validation cross-checked 90 plan rows, 90 result rows, and 180 controller/payload raw logs. Frozen JSON summaries and validations are identified by SHA-256. Table-generation scripts verify input hashes before deterministic projection. Analysis is descriptive: counts, observed proportions, median, inclusive-linear quartiles, IQR, minimum, and maximum. No hypothesis test, confidence claim, or between-mode ranking is used.

IV. SETUP-BOUNDED EVALUATION

A. Nominal Observations

A 65-s nominal (N0) window preceded the primary campaign and another followed it. Each contained 13 ONLINE records. The successful-message counter rose strictly from 1835 to 1955 before the campaign and from 3049 to 3169 after it. A separate 605-s nominal observation contained 121 ONLINE records and rose from 1760 to 2960. Timeout, CRC, sequence, and recovery counter deltas were zero in all three observations, and their prohibited-marker lists were empty (Table II). Only within-observation changes are interpreted; N0 is not a comparative baseline or a reliability estimate.

B. Primary Outcomes and Host-Observed Timing

All 90 planned trials were valid. Each mode had 30 confirmed activations and 30 confirmed NORMAL restorations. The predefined detector and recovery markers appeared in

TABLE III
PRIMARY OBSERVED OUTCOMES ON ONE G431/G474 PAIR.

Mode	Valid	Detect	Recover	Offline	Start
SILENT	30/30	30/30	30/30	30/30	0/30
BAD_CRC	30/30	30/30	30/30	30/30	0/30
DELAYED	30/30	30/30	30/30	30/30	0/30

TABLE IV
HOST-OBSERVED COMMAND-TO-MARKER LATENCY (MS), $n = 30$ PER ROW.

Mode	Int.	Median	Q1	Q3	IQR	Min-Max
SILENT	Det.	383.0	288.75	485.0	196.25	125–578
BAD_CRC	Det.	359.5	171.25	464.25	293.0	78–531
DELAYED	Det.	406.0	297.0	512.0	215.0	156–610
SILENT	Rec.	297.0	144.25	375.0	230.75	78–516
BAD_CRC	Rec.	359.0	113.75	417.25	303.5	47–531
DELAYED	Rec.	296.5	187.25	425.5	238.25	94–531

30/30 valid trials per mode (observed proportion 1.0). OFFLINE appeared in 30/30 trials per mode during the 4-s fault window, including BAD_CRC where it was not the detector. No post-injection link-start marker appeared; this absence is not reset evidence (Table III).

Host-observed command-to-detector-marker medians were 383.0 ms for SILENT, 359.5 ms for BAD_CRC, and 406.0 ms for DELAYED. Restore-command-to-recovery-marker medians were 297.0, 359.0, and 296.5 ms, respectively. Table IV retains quartiles and observed ranges. These are host-path distributions, not controller-internal latency bounds.

The two BAD_CRC mechanism observations are separate from the 90-trial denominator. SHORT recorded CRC rejection, one timeout, and no OFFLINE before confirmed restoration. SUSTAINED recorded CRC rejection, timeouts at consecutive counts one and two, and OFFLINE at count three before confirmed restoration and recovery. This ordered trace supports the inspected control-flow explanation; it does not expand the campaign denominator.

C. Separate F411 Pair Campaigns

Two physical NUCLEO-F411RE controller/payload pairs each ran the same fixed F411 implementation with a 100-ms response deadline and predefined 12-row protocol: three normal controls and three rows at each of 90, 100, and 110 ms. Each pair retained 12/12 valid rows. For each pair, the three normal controls had no adverse marker; the three 90-ms rows accepted the delayed response without timeout, rejection, or OFFLINE; and all three rows at both 100 and 110 ms showed timeout, sequence rejection, and OFFLINE followed by confirmed restoration and recovery (Table V). No DELAYED response was accepted at 100 or 110 ms.

Here F/R denotes the predefined timeout/sequence-rejection/OFFLINE pattern plus confirmed restoration and recovery, not a pooled metric. Post-activation NORMAL accepts were retained as boundary/pipeline observations in four Pair-1 rows and three Pair-2 rows. In every 100- and 110-ms row, cumulative timeout and sequence counter

TABLE V
F411 OUTCOMES WITH SEPARATE PAIR DENOMINATORS.

Pair	Valid	NC clean	90-ms accept	100-ms F/R	110-ms F/R
Pair-1	12/12	3/3	3/3	3/3	3/3
Pair-2	12/12	3/3	3/3	3/3	3/3

deltas were eight each, while explicit markers comprised two timeout transitions and one sequence rejection; counters and explicit markers therefore remain separate. The repeated pair-specific pattern supports protocol-level portability under these configurations, not independent replication of the primary pair, G431/G474 timing replication, pooling, MCU equivalence, or population inference.

V. DISCUSSION AND VALIDITY BOUNDARIES

Two-stage confirmation gating changes what a scored result means. A detector count is conditional on payload confirmation of configured fault-mode entry, and a recovery count is conditional on payload confirmation of return to NORMAL; neither confirmation is an independent measurement of the corresponding UART behavior on the wire. Missing activation confirmation excludes the trial from both detection and recovery scoring. With confirmed activation but missing restoration, detection remains scoreable while recovery is invalid and unscored. The five-event record makes these eligibility boundaries explicit; exact-byte traces and hashes support their audit. The primary campaign demonstrates that the complete requested-to-recovered chain can be operated and audited on the evaluated G431/G474 setup. It demonstrates protocol operability and auditability, but does not estimate how often request-only scoring would produce a different result because no ungated comparative baseline was run. The N0 windows are setup-integrity observations, not interleaved no-fault treatment controls; they therefore neither establish a comparative fault effect nor quantify natural system variance. The evaluation instead asks whether predefined controller outcomes occur after separately confirmed endpoint conditions; comparative causal attribution would require an interleaved control design beyond the present protocol evaluation. The separate F411 campaigns demonstrate the same confirmation rules on two further pair-specific setups. Neither result estimates how often unobserved devices would behave similarly.

Construct validity is bounded by observable endpoint behavior and predefined log markers. SILENT, BAD_CRC, and DELAYED do not represent all communication faults and are not equivalent to electrical, environmental, or radiation-induced faults. Markers operationalize detection and recovery but do not expose every internal state. Literal heartbeat or watchdog strings are not treated as independent health measurements.

Internal validity benefits from a confirmed NORMAL precondition, seeded order and offsets, fixed observation windows, separate endpoint logs, and stage-specific eligibility rules for missing confirmations. Sequential trials still share hardware, host, thermal, state-history, and timing conditions. The two

65-s N0 windows, separate 605-s observation, and one 65-s gate per F411 pair are duration bounded.

External and timing validity are similarly narrow. One primary pair and two purposively evaluated F411 pairs do not define a device population. Other delay values, corruption patterns, intermittent behavior, concurrent loads, and recovery horizons remain untested. Host serial timestamps include transport and scheduling effects. Temperature, vibration, electromagnetic conditions, power transients, and radiation were not varied or measured. The evidence supports no environmental robustness, flight-qualification, mission-assurance, or complete-system claim.

VI. REPRODUCIBILITY

The retained primary package contains the fixed seed and run plan, per-trial rows, 180 byte-preserving raw logs, validation output, summaries, firmware provenance, and SHA-256 inventories. Separate frozen paths retain the 605-s observation, both BAD_CRC traces, and each F411 campaign. The manuscript references these existing records; it does not copy empirical evidence. Hash-checking generators project table inputs deterministically and reject a changed source. This workflow makes the path from endpoint bytes to reported values inspectable without treating a hash as evidence that the experimental construct is externally valid.

VII. CONCLUSION

This paper presented a two-stage confirmation-gated HIL trial protocol for payload-link supervision. A requested injection becomes scoreable only after separate payload-side confirmation of configured fault-mode entry; recovery becomes scoreable only after separate payload-side NORMAL confirmation. These confirmations establish scoring boundaries but do not independently measure corresponding UART behavior on the wire. Five distinct event classes operationalize those gates, and byte-preserving endpoint traces with hash-verified summaries support auditability. The primary 90-trial campaign, separate nominal and BAD_CRC observations, and two pair-separated F411 campaigns demonstrate the protocol only on the evaluated setups. The result is methodological and setup bounded, with no reliability, population, MCU-equivalence, or internal-timing inference.

ACKNOWLEDGMENT

During manuscript preparation, the authors used OpenAI ChatGPT, OpenAI Codex, and Anthropic Claude for language editing and drafting support throughout Sections I–VII, literature-search and synthesis assistance for Section II, and code/tooling assistance for evidence validation described in Section VI. The authors checked all cited sources, technical claims, source-code behavior, experimental records, numerical results, and conclusions against primary sources or retained evidence. No AI-generated data were used. The authors take responsibility for the final manuscript.

REFERENCES

- [1] J. Arlat, M. Aguera, L. Amat, Y. Crouzet, J.-C. Fabre, J.-C. Laprie, E. Martins, and D. Powell, "Fault injection for dependability validation: A methodology and some applications," *IEEE Transactions on Software Engineering*, vol. 16, no. 2, pp. 166–182, 1990.
- [2] R. J. Martínez, P. J. Gil, G. Martín, C. Pérez, and J. J. Serrano, "Experimental validation of high-speed fault-tolerant systems using physical fault injection," in *Dependable Computing for Critical Applications 7*, 1999, pp. 249–265.
- [3] M. Abboush, D. Bamal, C. Knieke, and A. Rausch, "Hardware-in-the-loop-based real-time fault injection framework for dynamic behavior analysis of automotive software systems," *Sensors*, vol. 22, no. 4, p. 1360, 2022.
- [4] M. Abboush, C. Knieke, and A. Rausch, "A virtual testing framework for real-time validation of automotive software systems based on hardware in the loop and fault injection," *Sensors*, vol. 24, no. 12, p. 3733, 2024.
- [5] C. L. G. Batista, A. C. Weller, E. Martins, and M. de Fátima Mattiello-Francisco, "Towards increasing nanosatellite subsystem robustness," *Acta Astronautica*, vol. 156, pp. 187–196, 2019.
- [6] C. A. P. L. Conceição and M. de Fátima Mattiello-Francisco, "Systematic testing approach for communicating software embedded in nanosatellites focusing on interoperability faults," *Journal of Aerospace Technology and Management*, vol. 17, p. e1925, 2025.
- [7] M.-C. Hsueh, T. K. Tsai, and R. K. Iyer, "Fault injection techniques and tools," *Computer*, vol. 30, no. 4, pp. 75–82, 1997.
- [8] E. van der Kouwe, C. Giuffrida, and A. S. Tanenbaum, "Evaluating distortion in fault injection experiments," in *2014 IEEE 15th International Symposium on High-Assurance Systems Engineering*, 2014, pp. 25–32.
- [9] M. Short, M. J. Pont, and J. Fang, "Assessment of performance and dependability in embedded control systems: Methodology and case study," *Control Engineering Practice*, vol. 16, no. 11, pp. 1293–1307, 2008.
- [10] S. Blanc, A. Bonastre, and P. J. Gil, "Dependability assessment of by-wire control systems using fault injection," *Journal of Systems Architecture*, vol. 55, no. 2, pp. 102–113, 2009.